Tipos definidos por usuario, encapsulamineto

Programación II

06/0

8

Octubre

UNRN

Universidad Nacional de **Río Negro**

r27

Programación I

Tipos definidos por usuario I.

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Ejercicio: matriz rala

Definición

Una **matriz rala** es una matriz donde la **inmensa mayoría de sus casilleros contienen el valor cero** (0).

- **Casos de uso reales:** Grafos de redes sociales (matriz de adyacencia), procesamiento de lenguaje natural (matriz término-documento), imágenes astronómicas y simulaciones físicas.

Representación Tradicional (Densa)

Un arreglo bidimensional convencional `float M[10000][10000]`:

- Reserva **100 millones de posiciones** contiguas en RAM.
- Consume **~400 MB de memoria** , incluso si el 99.9% de los casilleros son ceros.
- **Resultado:** Desperdicio masivo de memoria procesando datos irrelevantes.

La Solución: Almacenamiento Disperso

En lugar de reservar toda la grilla, se guarda **únicamente la información de las celdas no nulas** mediante la coordenada y su valor:

```
Registro = (Fila, Columna, Valor)
```

Ejercicio: matriz rala

Pros y contras:

- **Eficiencia Espacial:**
  El consumo de memoria pasa de `filas x columnas` a **k**, donde k es la cantidad de elementos no nulos (`k << (filas x columnas)`).

- **Compromiso (*Trade-off*):**
  Ahorra un porcentaje enorme de memoria a cambio de aumentar complejidad algorítmica para buscar o insertar elementos (no hay acceso directo a la memoria como con los arreglos - A[i])

Tipos Abstractos de Datos

1. **Generalización:**
   Captar características y comportamientos comunes de problemas o entidades específicas, para formular una solución más abstracta y reutilizable (ya sea un TDA genérico o un algoritmo universal).

2. **Encapsulamiento:**
   Agrupar la definición y el comportamiento del tipo de datos en una sola entidad y esconder los detalles de implementación

3. **Separación de la responsabilidad:**
   Responsabilidad en un módulo y no desparramada en varios lugares.

Tipos Abstractos de Datos

1. **Ocultamiento de la información (encapsulamiento):**
   La idea clave es el *ocultamiento de la información*. El TDA mantiene escondidos los datos internos, y solo son accesibles mediante operaciones definidas en su *interfaz.*

   Esta idea permite escalar (crecer) las soluciones y asegurarse de que encajen al integrarse.

Componentes de un TAD

1. Los componentes mínimos de un TDA pertenecen a tres categorías:
   a. *Estado interno:* variables que guardan información.
   b. *Constructores: funciones para crear* una instancia.
   c. *Funciones de acceso:* devuelven información de la instancia. NO lo modifican.
   d. *Procedimientos de manipulación: modifican el objeto.*
   e. *Destructores: devuelven los recursos utilizados y destruyen la instancia.*

2. Por otro lado podemos construir bibliotecas de funciones. del TAD. La regla es que no deberían acceder directamente a la implementación sino que deben usar las funciones del TAD.

Implementar

```
// Tipo opaco. La abstracción
typedef struct MatrizRala MatrizRala_t;

// Constructor y Destructor
MatrizRala_t *matriz_crear(size_t filas, size_t columnas);
void matriz_destruir(MatrizRala_t **matriz_ptr);

// Operaciones de acceso y modificación
bool matriz_asignar(MatrizRala_t *matriz, size_t f, size_t c, float valor); float
matriz_obtener(const MatrizRala_t *matriz, size_t f, size_t c);

// Copia Profunda
MatrizRala_t *matriz_clonar(const MatrizRala_t *origen);

// Utilidades
size_t matriz_obtener_cantidad_no_ceros(const MatrizRala_t *matriz);
void matriz_imprimir(const MatrizRala_t *matriz);
```

Implementar

```
// Tipo opaco
typedef struct Matri

// Constructor y Des
MatrizRala_t *matriz                                    mnas);
void matriz_destrui

// Operaciones de ac
bool matriz_asignar(                                    size_t c, float valor); float
matriz_obtener(const                                    , size_t c);

// Copia Profunda
MatrizRala_t *matriz_c                                  ;

// Utilidades
size_t matriz_obtener_ca                  _ceros(const MatrizRala_t *matriz);
void matriz_imprimir(const MatrizRala_t *matriz);
```

